

ENGINEERING REBOOT 2026

Week 03 - Data Structures & Collection Processing

Day 14 Field Manual

Collection Processing & Analysis

Version 1.1 - Re-Baselined Edition - Locked Publication Edition

Status: LOCKED

PHASE 0 - MISSION BRIEF

Objective

Learn how software systems process collections of information to generate meaningful reports

During Days 11-13, collections were primarily used for storing information.

Day 14 introduces the next logical step:

Collection Processing & Analysis

The goal is no longer simply storing data.

The goal is extracting useful information from stored data.

Core Concepts

- Collection Processing
- Aggregation
- Reporting
- `sum()`
- `len()`
- `min()`
- `max()`
- Counting Patterns
- Data Analysis

Engineering Applications

Sales Reporting

```
sales = [120, 250, 90, 310, 175]
```

Questions:

- What are total sales?
- What is the average sale?
- What is the highest sale?
- What is the lowest sale?

Student Performance Analysis

```
scores = [88, 92, 76, 81, 95, 67]
```

Questions:

- What is the average score?
- What is the highest score?
- What is the lowest score?
- How many students passed?

Environmental Monitoring

```
temperatures = [72, 75, 71, 78, 80, 74, 69]
```

Questions:

- What is the average temperature?
- What is the highest temperature?
- What is the lowest temperature?

Learning Objectives

By the end of Day 14 you should be able to:

- Process numerical collections
- Calculate totals
- Calculate averages
- Determine minimum values
- Determine maximum values
- Count records meeting criteria
- Generate summary reports
- Apply collection-processing patterns
- Prepare for Day 15 capstone work

Deliverable Inventory

1. sales_report.py
2. score_processor.py

3. temperature_analysis.py

4. Notes.md

5. Journal.md

6. Evidence Package

PHASE 1 - SALES REPORT

Filename

`sales_report.py`

Implement

Create:

```
sales = [120, 250, 90, 310, 175]
```

Calculate:

- Total Sales
- Average Sale
- Highest Sale
- Lowest Sale

Display results in report format.

Suggested Processing

Calculate total sales:

```
total_sales = sum(sales)
```

Calculate average sale:

```
average_sale = total_sales / len(sales)
```

Determine highest sale:

```
highest_sale = max(sales)
```

Determine lowest sale:

```
lowest_sale = min(sales)
```

Validation Matrix

Expected Output

Sales Report

Total Sales: 945

Average Sale: 189.0

Highest Sale: 310

Lowest Sale: 90

Evidence Checkpoint

Capture:

sales_report-Code.png

Capture:

sales_report-Output.png

Completion Checkpoint

■ Exercise Completed

■ Validation Completed

■ Evidence Captured

PHASE 2 - SCORE PROCESSOR

Filename

`score_processor.py`

Objective

Analyze a collection of student scores and generate a summary report.

This exercise introduces an important collection-processing pattern:

Statistical Analysis + Conditional Counting

Implement

Create:

```
scores = [88, 92, 76, 81, 95, 67]
```

Calculate:

- Average Score
- Highest Score
- Lowest Score

Count:

- Passing Students (70 and above)

Display results in report format.

Suggested Processing

Calculate average score:

```
average_score = sum(scores) / len(scores)
```

Determine highest score:

```
highest_score = max(scores)
```

Determine lowest score:

```
lowest_score = min(scores)
```

Count passing students:

```
passing_count = 0

for score in scores:

    if score >= 70:

        passing_count += 1
```

Validation Matrix

Expected Output

Score Report

Average Score: 83.17

Highest Score: 95

Lowest Score: 67

Passing Students: 5

Challenge Extension

Calculate:

Students Scoring 90+

Suggested Logic

```
high_score_count = 0

for score in scores:

    if score >= 90:

        high_score_count += 1
```

Evidence Checkpoint

Capture:

score_processor-Code.png

Capture:

score_processor-Output.png

Completion Checkpoint

- Exercise Completed
- Validation Completed

■ Evidence Captured

PHASE 3 - TEMPERATURE ANALYSIS

Filename

temperature_analysis.py

Objective

Analyze a collection of temperature readings and generate a summary report.

This exercise reinforces:

- Aggregation
- Statistical Analysis
- Collection Traversal
- Conditional Counting

Implement

Create:

```
temperatures = [72, 75, 71, 78, 80, 74, 69]
```

Calculate:

- Average Temperature
- Highest Temperature
- Lowest Temperature

Display results in report format.

Suggested Processing

Calculate average temperature:

```
average_temp = sum(temperatures) / len(temperatures)
```

Determine highest temperature:

```
highest_temp = max(temperatures)
```

Determine lowest temperature:

```
lowest_temp = min(temperatures)
```

Validation Matrix

Expected Output

Temperature Analysis

Average Temperature: 74.14

Highest Temperature: 80

Lowest Temperature: 69

Challenge Extension

Calculate:

Temperatures Above Average

Suggested Logic

```
above_average_count = 0
```

```
for temp in temperatures:
```

```
    if temp > average_temp:
```

```
        above_average_count += 1
```

Display:

Temperatures Above Average: X

Validation Matrix - Challenge Extension

Expected Result:

Temperatures Above Average: 3

Evidence Checkpoint

Capture:

temperature_analysis-Code.png

Capture:

temperature_analysis-Output.png

Completion Checkpoint

■ Exercise Completed

■ Validation Completed

■ Evidence Captured

PHASE 4 - NOTES

Create

03-Week-03/Day-14/Notes/Notes.md

Objective

Document concepts learned during Day 14.

Required Sections

- Concepts Learned
- Key Commands
- Key Observations
- Lessons Learned
- Questions For Future Study

Suggested Topics

What is collection processing?

What is aggregation?

How does `sum()` work?

How does `len()` work?

How are averages calculated?

How are minimum values determined?

How are maximum values determined?

How can records be counted using loops?

Where might collection processing appear in real-world software?

Notes Template

Concepts Learned

Key Commands

Key Observations

Lessons Learned

Questions For Future Study

Completion Checkpoint

■ Notes Completed

PHASE 5 - JOURNAL

Create

03-Week-03/Day-14/Journal/Journal.md

Objective

Capture personal reflections regarding Day 14 execution.

Required Sections

- Summary
- Challenges Encountered
- Solutions Applied
- Confidence Assessment
- Reflection
- Tomorrow Preparation

Suggested Reflection Questions

How does collection processing differ from simply storing data?

Which exercise felt most useful?

Which concept felt easiest?

Which concept felt most challenging?

Where do you think collection processing appears in professional software systems?

How prepared do you feel for Day 15?

Journal Template

- # Summary
- # Challenges Encountered
- # Solutions Applied
- # Confidence Assessment
- # Reflection
- # Tomorrow Preparation

Completion Checkpoint

- Journal Completed

PHASE 6 - GIT OPERATIONS

Objective

Preserve Day 14 artifacts in source control and verify repository synchronization.

Execute

```
git status
```

```
git add .
```

```
git commit -m "Complete Day 14 collection processing exercises"
```

```
git push
```

```
git fetch
```

```
git status
```

```
git branch -vv
```

Preferred End State

Your branch is up to date with 'origin/main'

nothing to commit, working tree clean

Evidence Checkpoint

Capture:

Git-Operations.png

Must Show

```
git add .
```

```
git commit
```

```
git push
```

```
git fetch
```

```
git status
```

```
git branch -vv
```

Completion Checkpoint

■ Git Operations Completed

■ Repository Synchronized

■ Evidence Captured

END-OF-DAY SUCCESS CRITERIA

- sales_report.py completed
- score_processor.py completed
- temperature_analysis.py completed
- All validation completed
- Exercise evidence captured
- Notes completed
- Journal completed
- Git workflow completed
- Git-Operations.png captured
- Remote synchronization verified
- Working tree clean

MISSION COMPLETE ✓

Congratulations.

You have successfully completed Day 14:

Collection Processing & Analysis

You have now demonstrated the ability to:

- Process collections
- Generate statistical summaries
- Calculate averages
- Determine minimum values
- Determine maximum values
- Count matching records
- Produce structured reports

DAY 15 PREVIEW

Topic

Personal Data Analyzer

Expected Deliverable

`personal_data_analyzer.py`

Core Concepts

- Lists
- Dictionaries
- Collection Traversal
- Collection Processing
- Aggregation
- Reporting

Engineering Focus

Combine all Week 03 concepts into a single capstone application.

The Day 15 project will analyze structured personal data and generate useful reports using